



UNIVERSIDAD TECNOLÓGICA DEL NORTE DE AGUASCALIENTES

Rincón de Romos, Aguascalientes

ACTIVIDAD DE INVESTIGACIÓN

Conceptos relacionados con LLM

ASIGNATURA

Fundamentos de Inteligencia Artificial

Token · Embedding · Transformer · Encoder · Decoder
Attention · Fine-Tuning

ALUMNO

Sergio Michell Carreón López

DOCENTE

Mtro. Gerardo Martínez

GRUPO

7A

CARRERA

ITIID

FECHA DE ENTREGA

6 de octubre de 2026

Conceptos relacionados con LLM

Investigación de los conceptos vistos en clase · Fundamentos de Inteligencia Artificial

Un LLM, por sus siglas en inglés *Large Language Model*, es un modelo de lenguaje grande. En palabras simples, es un programa que se entrenó leyendo muchísimo texto y con eso aprendió a completar ideas, contestar preguntas o redactar. Cuando usamos una herramienta como ChatGPT, detrás está este tipo de modelo.

En clase nos pidieron investigar siete conceptos que explican cómo trabaja: Token, Embedding, Transformer, Encoder, Decoder, Attention y Fine-Tuning. Aquí los explico con mis palabras y con un ejemplo. La tabla junta el nombre en inglés, cómo se dice en español y qué hace cada uno, porque si solo se memoriza el nombre es fácil confundirlos.

El recorrido se puede contar así. El texto se parte en pedazos, esos pedazos se vuelven números y entran a una arquitectura llamada Transformer. Adentro, la atención decide qué partes del texto pesan más. El encoder sirve para entender la entrada y el decoder para ir escribiendo la respuesta. El fine-tuning no ocurre en cada pregunta: es un ajuste previo para dejar el modelo mejor en una tarea concreta.

Concepto	En español	Qué hace
Token	Pedazo de texto	Parte el texto en unidades que el modelo puede leer. No siempre es una palabra completa.
Embedding	Vector o incrustación	Convierte cada token en una lista de números que representa su significado.
Transformer	Transformador	Es la arquitectura, el diseño de red, con el que están hechos los LLM actuales.
Encoder	Codificador	Lee la entrada completa y arma una representación de lo que el texto quiere decir.
Decoder	Decodificador	Genera la respuesta de un token a la vez.
Attention	Atención	Decide qué partes del texto pesan más en cada momento.
Fine-Tuning	Ajuste fino	Sigue entrenando un modelo que ya sabe, para dejarlo mejor en una tarea concreta.

La tabla es el mapa del trabajo. En las siguientes páginas desarrollo cada renglón.

1. Token

El token es el pedazo de texto con el que el modelo trabaja. No siempre coincide con una palabra. A veces es una palabra completa, a veces es solo un trozo y a veces es un signo, como una coma o un signo de interrogación.

El modelo no trae guardadas todas las palabras del idioma. Cuando se entrena, aprende pedazos que se repiten mucho y con esos pedazos arma las palabras. Por ejemplo, «jugando» puede quedar en dos partes, algo parecido a «jug» y «ando», porque «ando» sale en muchos verbos: cantando, caminando, estudiando.

El programa que hace este corte se llama tokenizador. Si escribo «¿Qué es un token?», el modelo no se queda con la pregunta completa. La corta, y a cada pedazo le pone un número que solo sirve para identificarlo. Ese número todavía no guarda el significado. De eso se encarga el embedding.

También existe un límite. Cada modelo acepta cierta cantidad de tokens al mismo tiempo; a eso se le llama ventana de contexto. Si el texto se pasa de esa ventana, el modelo ya no alcanza a verlo completo.

2. Embedding

El embedding es la lista de números que representa a un token. Esa lista se llama vector. El número de identificación solo dice cuál token es. El embedding intenta guardar cómo se usa y, con eso, algo de su significado.

Se entiende mejor con un mapa. Las palabras que aparecen en situaciones parecidas quedan cerca: «perro», «gato» y «mascota» estarían en la misma zona. «Perro» y «teclado» quedarían lejos. El mapa de un LLM no es de dos dimensiones. Un embedding puede tener cientos o miles de números, y la cercanía se calcula entre esas listas.

Con eso el modelo puede hacer operaciones con el lenguaje. No entiende como una persona, pero sí compara tokens y usa esa comparación para calcular qué podría seguir en la frase.

Hay un detalle más. El Transformer ve varios tokens al mismo tiempo, así que necesita el orden. Por eso al embedding se le suma la posición de cada token. Sin la posición, «Ana ve a Luis» y «Luis ve a Ana» se parecerían demasiado, aunque no dicen lo mismo.

3. Transformer

El Transformer es la arquitectura del modelo, o sea el diseño de la red neuronal. No es una pieza aparte de las demás: es la estructura donde viven la atención, el encoder y el decoder. Lo propusieron Vaswani y sus colegas en 2017, en el artículo *Attention Is All You Need*.

Los modelos anteriores leían el texto en fila, una palabra y después la otra. En una frase larga se les debilitaba lo que iba al principio. El Transformer trabaja de otra manera: relaciona las partes del texto usando atención y puede tomar en cuenta varios tokens a la vez.

GPT y BERT están contruidos como Transformers. No hacen el mismo trabajo, pero comparten esta arquitectura. La diferencia fuerte está en si usan encoder, decoder o los dos.

Por dentro, el Transformer repite varias capas. En cada capa ocurre la atención y después una red pequeña que mezcla la información de cada token. Con más capas, el modelo arma relaciones más complicadas. Por ejemplo, descubre quién hizo la acción, o qué palabra está negando a otra aunque estén separadas.

4. Encoder

El encoder es el codificador. Su trabajo es leer la entrada y convertirla en una representación de lo que el texto quiere decir. No se encarga de escribir la respuesta.

Para hacerlo ve el texto completo en las dos direcciones. Cuando analiza una palabra puede fijarse en lo que va antes y también en lo que va después. Por eso se dice que es bidireccional.

BERT es el ejemplo más conocido de un modelo que usa solo encoder (Devlin y colaboradores, 2019). Sirve para tareas de entender: clasificar si una opinión es positiva, buscar un párrafo parecido o señalar qué parte de un texto contesta una pregunta. No está pensado para ir armando una conversación token por token.

5. Decoder

El decoder es el decodificador. Es la parte que escribe. Genera la salida de un token a la vez. En cada paso calcula qué tokens pueden seguir y elige uno de los más probables. Ese token se agrega al texto y el paso se repite.

Hay una regla importante. Cuando va a predecir el siguiente token, el decoder no puede ver la respuesta que todavía no existe. Solo ve lo que ya escribió. Si el modelo también tiene encoder, además puede usar la representación de la entrada, por ejemplo la frase que hay que traducir.

La familia GPT funciona principalmente como decoder. El modelo descrito por Brown y colaboradores (2020) es de ese tipo. Por eso una respuesta se va formando pedazo por pedazo, aunque en pantalla la veamos de corrido.

No todos los LLM traen encoder y decoder juntos. Esta tabla los separa para no mezclar los nombres:

Tipo de modelo	Qué parte usa	Ejemplo	Para qué sirve más
Solo encoder	Encoder	BERT	Entender, clasificar o buscar en un texto.
Solo decoder	Decoder	GPT	Redactar, conversar y completar texto.
Encoder y decoder	Las dos	T5 y el Transformer de traducción	Pasar de un texto a otro, como traducir.

6. Attention

Attention quiere decir atención. Es el mecanismo con el que el modelo decide qué partes del texto importan más en cada momento. No todas las palabras pesan igual, y ese peso lo calcula la red: nadie lo escribe a mano.

Sirve un ejemplo. En la frase «Dejé la llave del carro sobre la mesa», cuando el modelo trabaja la palabra «llave» le conviene mirar «carro» y «mesa». Así sabe que se habla de la llave de un carro y del lugar donde quedó, no de una llave de agua. «Dejé» aporta la acción, pero ayuda menos a distinguir de qué llave se trata.

Se puede imaginar como un reparto. Para un token, el modelo reparte su atención entre los demás. Los que aportan al sentido reciben más y los que casi no cambian la idea reciben muy poco.

Esto no se hace una sola vez. El Transformer repite la atención en paralelo, y cada repetición se llama cabeza. Una cabeza puede fijarse en quién realiza la acción, otra en el objeto y otra en una palabra corta que cambia el sentido, como «no». Por eso una frase larga no se le desarma tan fácil.

La atención no es un programa separado del Transformer. Va por dentro del encoder y también por dentro del decoder. La tabla de abajo es solo una ilustración de la frase de la llave. No son porcentajes sacados de un modelo real.

Palabra	Cuánta atención recibe	Por qué
carro	Alta	Aclara que se habla de la llave del carro.
mesa	Alta	Dice el lugar donde se quedó.
sobre	Media	Une la llave con el lugar.
Dejé	Baja	Dice la acción, pero no de qué llave se trata.

Ejemplo armado para explicar la idea, cuando el modelo está leyendo la palabra «llave».

7. Fine-Tuning

Fine-tuning es el ajuste fino. Primero el modelo se entrena con una cantidad enorme de texto general. Esa etapa se llama preentrenamiento. Ahí aprende cómo se arma el idioma y también datos que venían en esos textos, pero todavía no queda especializado en un oficio.

El fine-tuning consiste en seguir entrenándolo un rato más, ahora con ejemplos de una tarea concreta. Pueden ser preguntas y respuestas de un tema, o mensajes ya marcados como spam y no spam. Con esos ejemplos se ajustan sus valores internos para que responda mejor en ese trabajo. Ese entrenamiento extra es mucho más corto que hacerlo desde cero.

No es lo mismo que escribirle una instrucción en el chat. La instrucción, el prompt, orienta una respuesta sin cambiar lo que el modelo ya trae aprendido. El fine-tuning sí modifica el modelo. Por eso necesita ejemplos y un entrenamiento, no solo un mensaje bien redactado.

Una forma sencilla de recordarlo: el preentrenamiento le enseña a usar el lenguaje. El fine-tuning le enseña la tarea.

Cómo se conectan

Los siete conceptos se entienden mejor si se ven en orden. Hay que tener cuidado con una cosa: no todos son un paso que ocurra cada vez que hacemos una pregunta.

1. **Token.** El texto se corta en pedazos.
2. **Embedding.** Cada pedazo se convierte en un vector y se le agrega su posición.
3. **Transformer.** Esos vectores entran a la arquitectura del modelo.
4. **Attention.** Adentro del Transformer se decide qué pedazos pesan más.
5. **Encoder.** Si el modelo lo trae, lee toda la entrada y la representa.
6. **Decoder.** Si el modelo lo trae, escribe la salida token por token.
7. **Fine-Tuning.** No pasa en cada pregunta. Es el ajuste que ya se hizo antes, con ejemplos de la tarea.

GPT, por ejemplo, no usa encoder: trabaja con decoder. BERT hace lo contrario. El Transformer original, el de Vaswani y colaboradores (2017), sí usa las dos partes. El encoder lee la frase de origen y el decoder escribe la frase de salida, como en una traducción.

Conclusión

Token y embedding preparan el texto para que una computadora pueda trabajar con él. El Transformer lo procesa, y la atención es la pieza que relaciona unas palabras con otras aunque no estén juntas. El encoder se orienta a entender la entrada y el decoder a producir la salida. El fine-tuning llega cuando ese modelo general se quiere dejar listo para un trabajo concreto.

Vistos de esta forma, los nombres dejan de estar sueltos. Cada concepto responde una pregunta distinta: en qué se parte el texto, cómo se vuelve números, qué arquitectura lo procesa, qué parte entiende, qué parte escribe, cómo elige a qué mirar y cómo se especializa.

Para ubicarlos rápido, esta tabla va al revés: primero la idea y después el nombre.

Si preguntan esto	Concepto	En español
En qué pedazos se parte el texto	Token	Pedazo de texto
Cómo se guarda el significado con números	Embedding	Vector o incrustación
Cuál es el diseño de la red	Transformer	Transformador
Qué parte lee la entrada	Encoder	Codificador
Qué parte escribe la respuesta	Decoder	Decodificador
Cómo se decide qué palabras mirar	Attention	Atención
Cómo se especializa en una tarea	Fine-Tuning	Ajuste fino

Referencias

- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). *Language models are few-shot learners*. <https://arxiv.org/abs/2005.14165>
- Devlin, J., Chang, M.-W., Lee, K. y Toutanova, K. (2019). *BERT: Pre-training of deep bidirectional transformers for language understanding*. <https://arxiv.org/abs/1810.04805>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. y Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30. <https://arxiv.org/abs/1706.03762>